

Projet

Cheminement optimal dans les graphes

1 Objectifs

Le but de ce projet est d'implémenter efficacement l'algorithme de Dijkstra permettant de calculer la plus courte chaîne entre deux sommets dans un graphe non-orienté, sans boucle, ainsi que des variantes qui permettent de diminuer le nombre de sommets visités sur des graphes de grande taille. Une structure de tas sera utilisée pour la mise en œuvre de l'algorithme de Dijkstra.

2 Algorithmes

2.1 Algorithme de Dijkstra

Nous rappelons dans ce qui suit la description de l'algorithme de Dijkstra (dans le cas non orienté).

Algorithme de Dijkstra

Soit un graphe non orienté $G = (V, A)$ muni d'un système c de poids positifs sur les arêtes.

On note uv une arête de G et c_{uv} son poids. On note, pour u sommet de V , $V(u)$ l'ensemble des voisins de u .

On prend π un vecteur réel sur les $n = |V|$ sommets du graphe.

On pose T un vecteur d'entiers sur les n sommets du graphe.

On appelle ici "sommet marqué" un sommet qui a déjà été traité.

Soit un sommet r quelconque que l'on appellera *racine*. L'algorithme suivant calcule l'arbre des plus courts chemins reliant r à tous les sommets accessibles dans G à partir de r .

$\forall u \in V, \pi(u) = +\infty$ et $T(u) = -1$

$\pi(r) = 0$

$S \leftarrow \{r\}$

Tant que S non vide FAIRE

Prenons $s \in S$ tel que $\pi(s)$ soit minimum

$S \leftarrow S \setminus \{s\}$

Marquer s

Pour tout $u \in V(s)$ tel que u non marqué

$\pi(u) \leftarrow \min(\pi(u), \pi(s) + c_{su})$

$T(u) \leftarrow v$, où v est le voisin de u dans le plus court chemin connu de u à s .

$S \leftarrow S \cup \{u\}$

FIN Tant que

SORTIE : Pour tout sommet $v \in V$, $\pi(v)$ donne la valeur des plus courtes chaînes entre u et r .

A la fin de cet algorithme, si un sommet u n'est pas accessible depuis r , $\pi[u]$ est égale à l'infini et $T[u] = -1$. Dans le cas contraire, le tableau T permet de reconstituer la plus courte chaîne menant de r à u .

2.2 Améliorations

On s'intéresse à une façon de minimiser le nombre de sommets visités lors de la recherche de la plus courte chaîne entre un sommet origine o et un sommet destination p .

Algorithme A*

On associe à chaque sommet x du graphe une fonction d'évaluation par défaut $h(x)$ du coût de la plus courte chaîne de x à p . Une modification de l'algorithme de Dijkstra tenant compte de cette fonction d'évaluation permettrait d'élaguer un certain nombre de sommets à examiner à chaque itération.

Il s'agira de choisir à chaque itération le sommet d'approximation f minimale avec $f(x) = h(x) + \pi(x)$ pour tout sommet x .

L'algorithme serait alors :

$\forall u \in V, \pi(u) = +\infty, f(u) = h(u)$ et $T(u) = -1$

$\pi(r) = 0,$

$s \leftarrow r$

Tant que $s \neq p$ FAIRE

Marquer s

Pour tout $u \in V(s)$ tel que u non marqué

$\pi(u) \leftarrow \min(\pi(u), \pi(s) + c_{su})$

$f(u) \leftarrow \pi(u) + h(u)$

$T(u) \leftarrow v$, où v est le voisin de u dans le plus court chemin connu de u à s .

Prenons s non marqué tel que $f(s)$ soit minimum

FIN Tant que

SORTIE : Pour tout sommet $v \in V, \pi(v)$ donne la valeur des plus courtes chaînes entre u et r .

Sur les instances proposées, une fonction d'évaluation par défaut d'un sommet x serait la distance euclidienne entre x et la destination p pour l'algorithme A*.

Recherche bi-directionnelle

Une autre variante de l'algorithme de Dijkstra pour le calcul d'une plus courte chaîne entre deux sommets s et p consiste à dérouler alternativement l'algorithme à partir de s dans G (recherche avant) et à partir de p dans le graphe inverse (identique à G dans le cas non-orienté) (recherche arrière). Deux vecteurs réels π^{av} (resp. π^{ar}) sur les $n = |V|$ sommets du graphe seront utilisés pour mémoriser la valeurs des plus courtes chaînes pour la recherche avant (resp. arrière).

Lorsqu'une arête $\{u, v\}$ est examinée lors d'une recherche avant et que le sommet v a déjà été visité lors d'une recherche arrière, les valeurs des plus courtes chaînes de s à u et de v à p sont déjà calculées. Si $\pi^{av}(p) > \pi^{av}(u) + c_{uv} + \pi^{ar}(v)$, la valeur de $\pi^{av}(p)$ est mise à jour car une chaîne plus courte que celles déjà calculées vient d'être trouvée. Un raisonnement similaire peut être effectué pour les recherches arrières. L'algorithme s'arrête lorsque la recherche dans une direction examine un sommet déjà examiné dans l'autre direction. On utilise une stratégie qui permet d'équilibrer les recherches avant et arrière.

2.3 Mise en œuvre

Il s'agit d'implémenter l'algorithme de Dijkstra ainsi que les variantes proposées pour minimiser le nombre de sommets visités. Le choix du langage de programmation retenu est le langage Java. Il s'agira de comparer le nombre de sommets visités pour l'ensemble des algorithmes mis en œuvre ainsi que le temps de calcul mis pour calculer une solution optimale.

3 Structure de données

3.1 Graphes

On codera un graphe non orienté par une table de hachage qui associe à chaque sommet x la liste des couples (y, c_{xy}) .

3.2 Tas

Dans les algorithmes que vous aurez à mettre en œuvre, vous utiliserez un Tas pour gérer la bordure des sommets ouverts à chaque itération. La priorité d'un sommet correspondra à l'évaluation (ou l'approximation) minimale.

4 Instances

Pour tester au mieux la capacité de vos algorithmes dans des cas d'applications pratiques, vous l'utiliserez pour des instances réelles.

Pour cela, il faudra considérer les instances de la TSPLIB qui sont en fait des listes de coordonnées de points sur un plan. Elles proviennent de cartes géographiques ou de problèmes pratiques (circuits intégrés,...). Plus particulièrement, il faut considérer les instances zippées de la partie "Symmetric traveling salesman problem" du site :

<https://github.com/shredderzwj/TSPLIB/tree/master/res>.

Les graphes à tester seront construits à partir de ces instances et en fonction d'un paramètre p . En fait, le graphe déduit de l'instance possède l'arête entre deux points i et j si la distance euclidienne

$$\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

est inférieure à $p\%$ de la plus grande distance entre deux points de l'instance.

Pour effectuer les comparaisons entre les algorithmes, il faudra considérer les instances de grande taille de la TSPLIB.

Vous pourrez également écrire un programme pour générer des instances de graphes euclidiens en choisissant aléatoirement des sommets (points distincts) à l'intérieur d'un carré.

5 Jeux d'essais

Vous produirez plusieurs jeux d'essais pour tester les capacités de votre code.

En particulier, vous calculerez des plus courtes chaînes (attention les sommets sont numérotés de 0 à $n - 1$)

- att48 (48 capitales d'état des Etats Unis) pour $p = 15\%$, entre les points 43 et 45 (New-York et Seattle).
- att48 pour $p = 12\%$ et les mêmes points.
- a280 pour $p = 6\%$ et les points 8 et 92.

Mais évidemment, vous pouvez multiplier les exemples de manière à cerner les capacités de votre implémentation.

6 Bonus : Visualisation

Vous fournirez un outil de visualisation des instances et des solutions.

7 Organisation et dates

Le projet peut être réalisé en binôme et doit être rendu par mail à **eric.soutil@lecnam.net** au plus tard le **30/06/2026** :

- Partie 1** : Implémentation de la structure de données Tas avec les primitives de base dont la recherche du minimum, la modification de la priorité d'un sommet, la suppression d'un sommet et l'ajout d'une priorité dans le tas.
- Partie 2** : Implémentation des algorithmes proposés. Votre livraison sera constituée d'une archive qui doit comporter un rapport décrivant votre implémentation et votre code ainsi que la description des tests de validation et les conclusions concernant la comparaison des algorithmes (courbes et statistiques pour évaluer les performances des différents algorithmes) sur un benchmark d'instances.